

Platform Engineer Assignment

Cloud Infrastructure and Application Delivery Challenge

Terraform, Ansible, Docker, GitLab CI/CD, AWS ECS, DevSecOps

Objective

Design, build, and operate an automated pipeline that provisions AWS infrastructure with Terraform, configures compute with Ansible, containerizes a real application you build yourself, and deploys it to Amazon ECS through a GitLab CI/CD pipeline. The pipeline must support both automated deployment and dynamic vertical scaling of the running service, and the whole exercise must be backed by professional grade documentation, exactly as you would be expected to deliver in a production platform engineering role.

Scenario

A product team is launching “TaskFlow”, a small internal task management service. You are the platform engineer responsible for taking it from source code to a running, observable, autoscaling AWS deployment, and for leaving behind documentation good enough that another engineer could operate it without asking you a single question.

How the sections relate

Sections 1 and 2 are two halves of one deliverable: the application you build in Section 2 is the exact workload that gets containerized, provisioned, configured, and deployed by Section 1. Sections 3 and 4 apply across both.

1. Infrastructure, Automation and CI/CD Pipeline

Terraform, Ansible, Docker, and GitLab CI/CD, used together to build and operate a real deployment pipeline.

1.1 Containerization

- Write a production quality Dockerfile for the application from Section 2: multi-stage build, non-root user, minimal base image, no secrets baked into layers.
- Push versioned images (commit SHA tag, not just “latest”) to Amazon ECR.

1.2 Infrastructure as Code: Terraform

Define, in modular Terraform, everything required to run the service in a private network behind a public load balancer:

- A VPC with public and private subnets across at least two Availability Zones, a NAT gateway, and route tables.
- An ECS cluster on EC2 (an Auto Scaling Group of ECS-optimized instances, not Fargate), so that Ansible has real hosts to configure. Justify this choice in your ADR (Section 3.4).
- An Application Load Balancer, target group, and listener routing to the ECS service.
- An ECS task definition and service for the TaskFlow application, plus its ECR repository.
- IAM roles scoped by least privilege: an EC2 instance role, an ECS task execution role, and an ECS task role.
- A managed database (RDS PostgreSQL) in the private subnets, with credentials stored in AWS Secrets Manager and injected into the task definition. Never hard-code credentials.

- CloudWatch log group(s) for both the EC2 instances and the ECS service, with an explicit retention policy.
- **Vertical scaling, implemented for real:** build a mechanism that changes the task definition's CPU/memory allocation and forces a new deployment in response to sustained CloudWatch CPU/memory utilization, for example a CloudWatch alarm invoking a small Lambda (or a CI-triggered job) that runs a scoped Terraform apply. Manually editing the task definition in the console does not satisfy this requirement.
- Bonus: add ECS Service Auto Scaling (target tracking) for horizontal scaling alongside your vertical mechanism, and explain in the runbook when each one applies.

1.3 Configuration Management: Ansible

Use Ansible to configure the EC2 instances that Terraform provisions. This is what makes the exercise reflect a real IaC and configuration management split:

- A dynamic inventory sourced from AWS (the `aws_ec2` plugin) that picks up instances by the tags Terraform applies, with no hard-coded IP lists.
- Role(s) that install and configure Docker, the CloudWatch agent, and the ECS container agent settings.
- Baseline OS hardening: automatic security updates, a minimal firewall policy, and disabled password SSH authentication.
- Playbooks must be idempotent: running them twice must produce no changes on the second run. Capture this as evidence.

1.4 GitLab CI/CD Pipeline

Build a `.gitlab-ci.yml` with at least the stages below. Protect the apply stage behind a manual gate on your default branch, and mask and protect all AWS credentials as CI/CD variables.

```
stages:
  - lint          # terraform fmt/validate, tflint, ansible-lint, app linter
  - security      # tfsec/checkov, trivy image scan, gitleaks secret scan
  - test          # unit tests + coverage gate
  - build         # docker build, tag with $CI_COMMIT_SHORT_SHA, push to ECR
  - plan          # terraform plan (artifact, posted as MR note)
  - apply         # terraform apply, manual gate, protected branch only
  - configure     # ansible-playbook against the new Auto Scaling Group
  - deploy        # ecs update-service --force-new-deployment
  - smoke-test    # curl /health through the ALB, fail pipeline on non-200
```

- Merge request pipelines run lint, security, test, build, and plan only, never apply.
- Terraform state must be stored remotely (for example S3 with a DynamoDB lock table), never committed to git.

1.5 Monitoring and Logging

- ECS service and EC2 instance logs shipped to CloudWatch Logs with a defined retention period.
- A CloudWatch alarm on CPU and memory utilization that drives the vertical scaling mechanism from Section 1.2.
- Optional bonus: a CloudWatch (or Grafana) dashboard showing request latency, task count, and resource utilization over time.

2. Software Engineering: Build the Application

Build the TaskFlow service yourself, in the language or framework of your choice (Node.js, Python, Go, Java, etc). This is the workload Section 1's infrastructure exists to run. It must actually be your own code, not a scaffold or tutorial clone.

2.1 Functional requirements

- A REST API for managing tasks: create, list, retrieve, update, and delete (full CRUD).
- Persistent storage in the RDS PostgreSQL instance provisioned in Section 1.2. No in-memory or file-based storage.
- A **/health** endpoint used by both the ALB target group health check and the ECS container health check.
- Input validation with meaningful 4xx error responses, and centralized error handling.
- An OpenAPI/Swagger specification describing the API.

2.2 Engineering practice requirements

- Configuration read entirely from environment variables (12-factor): database URL, log level, port, etc. Nothing environment-specific hard-coded.
- Structured (JSON) logging to stdout, so it flows cleanly into the CloudWatch log group from Section 1.5.
- A unit test suite with a minimum 70% coverage threshold, run as its own stage in the CI pipeline and failing the build below that threshold.
- Graceful shutdown handling (drain in-flight requests, close the DB pool) so ECS deployments and vertical scaling redeployments do not drop traffic.

2.3 Deployment target

The application built here is containerized per Section 1.1, deployed by the Terraform and Ansible provisioned infrastructure in Section 1.2 and 1.3, and released through the pipeline in Section 1.4. Treat these as one continuous delivery path, not two separate exercises.

3. Documentation

Documentation is graded as a first-class deliverable, not an afterthought. Everything below belongs in docs/ in your repository.

3.1 Architecture diagram

- One diagram (draw.io, Lucidchart export, or hand-drawn and scanned is fine) showing the VPC, subnets, ECS cluster, ALB, RDS, and how a request flows from the internet to the application and back.

3.2 README.md

At the repository root, including, carried over from the original assignment brief and still required:

- A short essay comparing DevOps vs. Platform Engineering.
- A discussion of the trending shift toward DevSecOps and why it matters.
- Setup and local run instructions, plus a summary of how to reproduce the deployment from a clean AWS account.

3.3 Operational Runbook (docs/runbook.md)

Written so that an on-call engineer who has never seen this system can follow it under pressure:

- Deployment procedure, step by step, including how to roll back a bad release.
- Scaling procedure: how the automated vertical scaling mechanism works, and how to intervene manually if it misbehaves.
- At least three incident scenarios with diagnosis and remediation steps, for example: ECS tasks stuck in PENDING, the ALB returning 502s, and database connection pool exhaustion.
- Backup and restore steps for the RDS database.

3.4 Architecture Decision Record(s) (docs/adr/)

- At least one ADR in standard format (Context / Decision / Consequences), covering at minimum why you hosted ECS on EC2 rather than Fargate for this exercise.

4. Version Control and Submission

4.1 Repository

Push all code, Terraform, Ansible, the application, the Dockerfile, the GitLab CI/CD YAML, and all documentation, to a single public GitHub or GitLab repository, structured as follows:

```
repo-root/
├── app/                                # Application source code and unit tests
│   ├── src/
│   ├── tests/
│   └── Dockerfile
├── terraform/                         # IaC: networking, ECS, RDS, IAM, CloudWatch
│   ├── modules/
│   └── envs/prod/
├── ansible/                          # Configuration management for ECS host instances
│   ├── inventory/
│   ├── roles/
│   └── site.yml
├── .gitlab-ci.yml                    # Pipeline definition
├── docs/
│   ├── architecture-diagram.png
│   ├── runbook.md
│   └── adr/
│       └── 0001-ecs-hosting-model.md
├── evidence/                        # Screenshots and recordings, see Section 4.2
└── README.md
```

4.2 Evidence directory

Place screenshots and/or short recordings in evidence/ that prove the system actually ran, not just that the code compiles:

- A full green pipeline run, all stages visible.
- terraform apply output (or the AWS Console) showing the created VPC, ECS cluster, ALB, and RDS instance.
- An Ansible playbook run, including a second, idempotent run showing zero changes.
- The running application responding through the ALB's public DNS name.
- CloudWatch logs showing application output.
- Evidence of a vertical scaling event actually firing: the CloudWatch alarm and the resulting task definition change, before and after.

4.3 Commit hygiene

- Commit history should show incremental, logically scoped work, not a single “initial commit” dump.
- Do not commit .tfstate files, .env files, private keys, or any AWS credentials. Use a .gitignore from the start.